Arquitetura ��� Sistema de Streaming de M��sica

Arquitetura de Software — Sistema de Streaming de Música

> Documento de referência técnica para a equipe (9 integrantes). Cobre requisitos, arquitetura, organização de pastas (MVC), integração entre os módulos e o setup do Windows Server 2019.

(Não consegui abrir o link do Google AI Mode que você enviou — o site bloqueia acesso automatizado. Se ele tiver algo importante que eu deva considerar, me manda o conteúdo direto que eu incorporo aqui.)

---

1. Visão geral

O sistema tem **3 frentes de cliente** e **1 back-end central**:

Camada	Tecnologia	Função
API + Web	Java Spring Boot (JPA, Web, Security, Mail, Thymeleaf, Driver JDBC)	Cérebro do sistema: autenticação, regras de negócio, streaming, e-mails, e também serve o cliente web para PC via Thymeleaf
App mobile	React Native (JavaScript)	Cliente principal do usuário final, com suporte a reprodução offline
Painel admin	PHP (MVC)	Gestão de catálogo (músicas, artistas, álbuns), usuários e relatórios
Infraestrutura	Windows Server 2019	Hospeda tudo: API, banco, arquivos de áudio e o admin

A ideia central: **a API Java é a única fonte de verdade**. O painel PHP não deve escrever direto no banco — ele consome endpoints administrativos da própria API. Isso evita duplicar regra de negócio em duas linguagens diferentes e evita inconsistência de dados. Mais detalhes na seção 6.

---

2. Requisitos do sistema

2.1 Funcionais

- Cadastro e login de usuário, com verificação de e-mail (Spring Mail)
- Recuperação de senha por e-mail (token com expiração)
- Catálogo: artistas, álbuns, faixas, playlists (criação/edição pelo usuário)
- Streaming de áudio com suporte a **HTTP Range Requests** (permitir avançar/retroceder sem baixar o arquivo inteiro)
- Download de faixas para **acesso offline** com controle de licença (seção 5)
- Sincronização de dados quando o app volta a ficar online (playlists criadas offline, histórico, etc.)
- Busca de músicas/artistas/álbuns
- Painel administrativo: CRUD de catálogo, gestão de usuários, upload de áudio, relatórios básicos de uso
- Cliente web (PC) com player embutido, renderizado via Thymeleaf
- Sistema de papéis: `USER`, `ADMIN` (e `PREMIUM`, se o projeto tiver planos pagos futuramente)
- Notificações por e-mail (boas-vindas, redefinição de senha)

2.2 Não funcionais

- **Segurança**: HTTPS obrigatório, senhas com BCrypt, autenticação via JWT para API/mobile, sessão via Spring Security para o cliente web
- **Disponibilidade**: API rodando como serviço do Windows com reinício automático em caso de falha
- **Performance**: streaming eficiente (Range Requests), cache de metadados, paginação em listagens
- **Consistência**: única fonte de verdade (API), nunca dois sistemas escrevendo direto no banco
- **Observabilidade**: logs em arquivo + Spring Actuator para health-check
- **Testabilidade**: testes unitários (JUnit/Mockito) no back-end, testes de API (Postman/Newman) para os fluxos críticos

---

3. Visão de arquitetura

```
flowchart TB
    subgraph Clients["Clientes"]
        RN["App Mobile — React Native"]
        WEB["Cliente Web — Thymeleaf"]
        ADM["Painel Admin — PHP"]
    end

    subgraph WinServer["Windows Server 2019"]
        IIS["IIS / Reverse Proxy (HTTPS :443)"]
        subgraph API["API Java Spring Boot"]
            SEC["Spring Security + JWT"]
            CTRL["Controllers (REST + MVC)"]
            SVC["Services / regras de negócio"]
            REPO["Repositories (JPA)"]
            MAIL["Email Sender"]
        end
        DB[("MySQL")]
        STORAGE[("Storage de áudio")]
    end

    RN -- "HTTPS / JSON (JWT)" --> IIS
    WEB -- "HTTPS (sessão)" --> IIS
    ADM -- "HTTPS / API interna (ROLE_ADMIN)" --> IIS
    IIS --> API
    API --> DB
    API --> STORAGE
    API --> MAIL
```

A API Java concentra tudo: ela é ao mesmo tempo uma API REST (para o mobile) e uma aplicação MVC com Thymeleaf (para o cliente web de PC). O PHP fica isolado como um cliente administrativo da API.

---

4. Estrutura de pastas — API Java (Spring Boot, MVC)

Sugiro manter o mesmo padrão de pacote dos outros projetos de vocês (`com.IEGP3...`), trocando só o nome do módulo:

```
com.IEGP3.streamingApp
├── config/            → SecurityConfig, WebConfig, CorsConfig, MailConfig
├── controller/
│   ├── rest/          → AuthController, TrackController, PlaylistController, SyncController (JSON, consumidos
pelo mobile)
│   ├── web/            → HomeController, PlayerController (retornam views Thymeleaf, para o cliente PC)
│   └── admin/          → endpoints só para ROLE_ADMIN, consumidos pelo PHP
├── model/              → entidades JPA: User, Role, Artist, Album, Track, Playlist, PlaylistTrack, Download,
```

```
PlayHistory
├── repository/         → interfaces JPA Repository
├── service/            → interfaces + implementação das regras de negócio
├── dto/                → request/response DTOs (nunca expor a entidade JPA direto no JSON)
├── security/           → JwtFilter, JwtUtils, UserDetailsServiceImpl
├── mail/                → EmailService + templates Thymeleaf de e-mail (boas-vindas, reset de senha)
├── exception/           → exceções customizadas + GlobalExceptionHandler (@ControllerAdvice)
├── util/                → StorageService (upload/leitura de áudio), TokenGenerator
└── StreamingAppApplication.java

resources/
├── templates/          → páginas Thymeleaf do cliente web + templates de e-mail
├── static/              → CSS/JS do cliente web
├── application.yml      → configs comuns
├── application-dev.yml
├── application-prod.yml  → sem senha em texto puro; usar variáveis de ambiente do Windows
└── db/migration/         → scripts Flyway (versionamento do banco)
```

Por que separar `controller/rest`, `controller/web` e `controller/admin`? Porque cada um tem uma preocupação de segurança diferente: o `rest` exige JWT, o `web` usa sessão de navegador, e o `admin` é só para quem tem `ROLE_ADMIN` (o PHP, nesse caso). Misturar tudo em um único pacote de controllers vira bagunça rápido com uma equipe de 9 pessoas mexendo ao mesmo tempo.

---

5. Acesso e reprodução offline

Esse é o ponto mais delicado do projeto. Fluxo recomendado:

1. Usuário pede o download de uma faixa no app → app chama `POST /api/v1/tracks/{id}/download-token`
2. API valida permissão do usuário e gera um **token assinado de curta duração** (JWT com `trackId`, `userId`, `exp`)
3. App usa esse token para baixar o arquivo de áudio (endpoint separado do streaming normal)
4. O arquivo é salvo **criptografado** no armazenamento local do celular, com uma chave gerada no primeiro login do dispositivo (nunca guardar o áudio "cru" no storage do app — facilita pirataria)
5. O app guarda localmente (SQLite/WatermelonDB) os metadados das faixas baixadas e a lista de IDs disponíveis offline
6. Periodicamente, quando o app tem conexão, ele **revalida a licença** com o servidor (ex.: a cada 7 dias). Se o usuário perdeu o acesso (ex.: cancelou assinatura, se houver plano pago), o app apaga a faixa local
7. Sincronização: mudanças feitas offline (nova playlist, faixa favoritada) ficam numa fila local e são enviadas ao servidor assim que a conexão volta (`POST /api/v1/sync`)

Isso não é DRM de verdade (não existe DRM 100% seguro em app), mas dá uma barreira razoável contra cópia trivial dos arquivos.

---

6. Integração Painel Admin (PHP) ↔ API Java

Modelo recomendado: o PHP não acessa o MySQL diretamente. Ele é um *cliente* da API, autenticado com uma conta de serviço (`ROLE_ADMIN`) via JWT ou API key fixa de uso interno. Ele chama endpoints como:

- `POST /api/v1/admin/tracks` (upload de nova música + metadados)
- `PUT /api/v1/admin/tracks/{id}`
- `GET /api/v1/admin/users`
- `GET /api/v1/admin/reports/usage`

Por que não deixar o PHP escrever direto no banco? Porque toda regra de validação, processamento de áudio (padronizar bitrate, por exemplo) e efeitos colaterais (invalidar cache, gerar

thumbnail) ficam centralizados num único lugar — a API. Se o PHP escrevesse direto, vocês teriam que reimplementar essas regras em PHP também, e as duas partes iam divergir com o tempo.

Se o prazo apertar e a equipe decidir que é rápido demais fazer tudo via API, o mínimo aceitável é: **catálogo simples (nome, descrição) pode ir direto ao banco, mas upload de áudio sempre passa pela API** — é ela quem sabe processar e guardar o arquivo corretamente.

Estrutura de pastas do painel PHP (mesmo padrão MVC que vocês já usam no SHINKA)

```
admin-panel/
├── app/
│   ├── controllers/
│   ├── models/          → aqui ficam os "clientes HTTP" que chamam a API (não models de banco)
│   ├── views/
│   ├── core/             → Router, ApiClient (wrapper do Guzzle), Auth
│   └── helpers/
├── public/
│   ├── index.php         → front controller
│   ├── css/
│   ├── js/
│   └── uploads/           → armazenamento temporário antes de enviar o arquivo pra API
├── config/
│   └── api.php            → URL base da API + credenciais da conta de serviço
└── vendor/                 → composer (Guzzle para chamadas HTTP)
```

---

7. Estrutura de pastas — App Mobile (React Native)

```
mobile-app/
├── src/
│   ├── api/             → cliente HTTP (axios), endpoints, interceptor de JWT
│   ├── components/       → componentes reutilizáveis de UI
│   ├── screens/           → Home, Player, Library, Downloads, Login, Profile
│   ├── navigation/         → React Navigation
│   ├── store/               → Redux Toolkit ou Zustand (estado de auth, player, downloads)
│   ├── services/
│   │   ├── downloadManager.ts   → download + criptografia local dos arquivos
│   │   └── audioPlayer.ts       → wrapper do react-native-track-player
│   ├── database/            → modelos locais (WatermelonDB/SQLite) para catálogo offline
│   ├── hooks/
│   └── utils/
├── android/
├── ios/
└── App.tsx
```

Bibliotecas que valem a pena olhar: `react-native-track-player` (player com suporte a background/lockscreen), `react-native-fs` (manipulação de arquivos), `@react-native-async-storage` ou WatermelonDB (dados locais).

---

8. Segurança

- HTTPS em tudo (sem exceção, nem entre PHP e API)
- Senha com BCrypt no banco, nunca texto puro
- JWT com tempo de expiração curto para o mobile + refresh token
- Sessão de servidor (cookie httpOnly) para o cliente web
- Endpoints `/admin/**` protegidos por `ROLE_ADMIN` e, de preferência, restritos por IP interno (o PHP roda no mesmo servidor, então pode bloquear esses endpoints para acesso só via `localhost`/rede interna)

- Rate limiting básico no login (evitar força bruta)
- Nunca commitar senha de banco/e-mail no `application.yml` — usar variáveis de ambiente do Windows

---

9. Windows Server 2019 — o que instalar

Item	Para quê
JDK 17 (ou 21 LTS) + `JAVA_HOME` configurado	Rodar a API Spring Boot
Maven (na máquina de build/CI, não precisa ser no servidor)	Compilar o projeto Java
MySQL Server 8.x (Community), instalado como serviço do Windows	Banco de dados
Role IIS (Web Server) + **PHP Manager for IIS** + **PHP 8.x (Non Thread Safe) via FastCGI**	Rodar o painel admin em PHP de forma nativa no Windows Server (alternativa mais "de produção" que o XAMPP que vocês usam nos outros projetos)
IIS Application Request Routing (ARR) + URL Rewrite	Fazer o IIS agir como *reverse proxy*: `/api/*` e `/` → Spring Boot (porta interna, ex. 8443), `/admin/*` → PHP — tudo atrás de uma única porta pública 443
Certificado TLS (Let's Encrypt via `win-acme`, ou certificado da instituição)	HTTPS
WinSW ou NSSM	Rodar o `.jar` do Spring Boot como **serviço do Windows** (inicia sozinho no boot, reinicia se cair)
Windows Defender Firewall — abrir só a porta 443 pra fora	Segurança de rede
Pasta dedicada (ex. `D:\StreamingApp\storage\audio\`) com permissão restrita à conta de serviço	Armazenamento dos arquivos de áudio
Agendador de Tarefas (Task Scheduler) rodando `mysqldump` + cópia da pasta de áudio	Backup periódico

> Se preferir simplicidade em vez de "produção de verdade", dá pra manter o XAMPP/Apache que vocês já usam no SHINKA e na biblioteca escolar — só que aí perde a integração nativa do IIS como proxy único. Funciona, mas exige rodar Apache e o serviço Java em portas separadas e configurar o roteamento manualmente (ou usar um reverse proxy leve como Nginx/Caddy no lugar do IIS).

---

10. Organização da equipe (9 pessoas)

Squad	Pessoas	Responsabilidade
Backend Java	2	API, Security, JPA, regras de negócio, e-mails
Mobile React Native	2	App, player, download offline, sincronização
Admin PHP	2	Painel administrativo, integração com a API
Infra / DevOps	1	Windows Server, IIS, banco, deploy, backup
QA	1	Testes de integração, roteiro de

		testes manuais
Tech Lead / coordenação	1	Arquitetura, revisão de PRs, alinhamento entre squads

---

11. Roadmap sugerido

1. **Fase 1 — Fundação**: modelagem do banco, autenticação (cadastro/login/JWT), CRUD básico de catálogo pela API
2. **Fase 2 — Streaming**: endpoint de streaming com Range Requests, player web (Thymeleaf) e player mobile básico (online)
3. **Fase 3 — Admin**: painel PHP consumindo a API, upload de áudio
4. **Fase 4 — Offline**: download com token, storage criptografado local, sincronização
5. **Fase 5 — Infra**: deploy no Windows Server 2019, IIS como proxy, HTTPS, backup automatizado
6. **Fase 6 — Polimento**: busca, playlists, relatórios de uso, testes finais

---

12. Colaboração em equipe — Git, IntelliJ e VSCode

Com 9 pessoas mexendo no mesmo projeto ao mesmo tempo, a peça central **não é o IntelliJ nem o VSCode** — é o **Git + um repositório remoto compartilhado** (GitHub, GitLab ou Azure DevOps). O editor que cada um usa é só uma questão de gosto pessoal; o que importa é todo mundo falando com o mesmo repositório.

12.1 Estrutura de repositórios

Com 3 módulos de tecnologias bem diferentes (Java, PHP, React Native), há duas opções:

- **Monorepo** (um único repositório com 3 pastas: `api-java/`, `admin-php/`, `mobile-app/`) — mais simples de coordenar para um time pequeno, todo mundo vê tudo, um único histórico
- **Multi-repo** (um repositório por módulo) — mais isolado, cada squad mexe só no seu, mas exige coordenar versões entre eles

Para uma equipe de 9 num projeto acadêmico/único, **monorepo é mais simples de gerenciar** e evita o trabalho de sincronizar 3 repositórios separados. Só cuidado para não deixar os `node_modules`, `target/` e `vendor/` do PHP indo pro Git (`.gitignore` bem feito).

12.2 Fluxo de branches (Git Flow simplificado)

- `main` → sempre estável, é o que está (ou vai) para o Windows Server
- `develop` → integração contínua do que cada squad termina
- `feature/nome-da-squad-o-que-e` → cada pessoa/dupla cria uma branch a partir de `develop` para cada tarefa (ex.: `feature/backend-auth-jwt`, `feature/mobile-player-offline`)
- Ninguém commita direto em `main` ou `develop` — tudo entra via **Pull Request** revisado por pelo menos 1 outra pessoa da squad
- Isso evita o clássico "duas pessoas editando o mesmo arquivo ao mesmo tempo e sobrescrevendo o trabalho uma da outra" — cada um trabalha isolado na própria branch, e o Git cuida de mesclar (ou avisa de conflito, que aí se resolve manualmente)

12.3 IntelliJ IDEA — configuração de Git

1. Ao clonar o repositório (`VCS → Get from Version Control`), o IntelliJ já detecta o Git automaticamente
2. Use o painel **Commit** (lado esquerdo) para revisar diffs antes de commitar
3. Use **Git → Branches** na barra inferior para trocar/criar branches sem precisar do terminal
4. Ative **File Watchers/Settings Sync** (opcional) para manter o formatador de código igual entre quem usa IntelliJ

12.4 VSCode — configuração de Git

1. A aba **Source Control** (ícone de ramificação na barra lateral) já mostra os arquivos alterados
2. Extensão **GitLens** ajuda bastante a ver quem alterou o quê e quando, útil numa equipe grande
3. Para o time do React Native/PHP que provavelmente vai usar VSCode, adicionem um `.editorconfig` no repositório para manter indentação/formatação consistente entre quem usa IntelliJ e quem usa VSCode

12.5 Edição em tempo real (opcional, além do Git)

Git resolve "trabalhar ao mesmo tempo sem pisar no trabalho um do outro" através de branches — não é edição simultânea no mesmo arquivo em tempo real. Se vocês quiserem literalmente ver o cursor de outra pessoa digitando ao vivo (útil pra pair programming ou tirar dúvida rápida), existem:

- **Code With Me** (plugin nativo da JetBrains, funciona no IntelliJ) — um convida, o outro entra pelo link, sem precisar instalar nada extra
- **Live Share** (extensão da Microsoft para VSCode) — mesma ideia, funciona entre VSCode e até com quem usa IntelliJ (Live Share tem suporte cross-IDE limitado)

Mas isso é complementar, não substitui o Git: usem o Live Share/Code With Me para sessões pontuais de pair programming, e o Git+PR para o fluxo do dia a dia.

12.6 Checklist prático para começar

1. Criar o repositório remoto (GitHub, por exemplo) e adicionar os 9 como colaboradores
2. Subir a estrutura inicial de pastas (seções 4, 6 e 7 deste documento) com um `.gitignore` por módulo
3. Proteger a branch `main` e `develop` (exigir Pull Request + pelo menos 1 aprovação antes de mesclar)
4. Cada um clona o repositório na própria máquina (IntelliJ para quem mexe em Java, VSCode para PHP/React Native — ou o que preferirem)
5. Definir a convenção de nome de branch e de commit (ex.: Conventional Commits: `feat:`, `fix:`, `docs:`)
6. Primeira reunião rápida pra alinhar quem pega qual branch/tarefa inicial

---

Se quiser, posso detalhar em outro documento o modelo de entidades do banco (com os relacionamentos) ou os contratos dos principais endpoints da API — me diga por qual desses dois vocês querem começar.